

P22 - Médian INF2 - durée : 1h30

documents autorisés : 1 feuille A4 recto-verso

Exercice 1 - Élections (copie n°1)

C'est le grand soir ! Tout le monde est allé voter ce dimanche et vous devez écrire un programme permettant de faciliter le dépouillement.

1.1. Préparer le dépouillement

Avant de procéder au dépouillement, il faut lister les candidats à l'élection. Disons qu'un ami s'en est chargé pour vous. De votre côté, vous avez besoin d'associer à chaque candidat un nombre de voix.

Écrire une fonction **creer_dictionnaire_candidats()** qui prend en entrée une liste des noms des candidats. Cette fonction retourne un dictionnaire dont les clés sont les noms des candidats et les valeurs le nombre de bulletins pour eux. Au début, le nombre de bulletins est à 0.

N'oubliez pas d'ajouter deux clés **Nul** et **Blanc** qui permettront de comptabiliser les bulletins nuls et blancs.

1.2. Dépouiller

Nous disposons désormais d'un dictionnaire de candidats associant leur nom à un nombre de bulletins. Procédons au dépouillement.

Écrire une fonction **lancer_depouillement()** qui prend en paramètre le dictionnaire des candidats. Cette fonction modifiera les valeurs du dictionnaire passé en paramètre et ne retournera rien.

La fonction **lancer_depouillement()** ne sera appelée qu'une seule fois dans le programme. Elle demandera en boucle à l'utilisateur d'entrer le nom du candidat présent sur le bulletin qu'il sort de l'urne et mettra à jour le dictionnaire des candidats. Si l'enveloppe est vide, l'utilisateur entrera **Blanc**.

Quelques points de vigilance :

- Vous ne connaissez pas à l'avance le nombre de bulletins contenu dans l'urne.
- Si le nom du candidat n'existe pas dans le dictionnaire, le bulletin comptera comme nul.

1.3. Enregistrer les résultats

Le dépouillement est terminé ! Vous disposez donc d'un dictionnaire comptabilisant les votes pour chaque candidat ainsi que les votes blancs et nuls.

Écrire une fonction **enregistrer_resultats()** qui prend en paramètres :

- Le nom de la ville **ville**
- Le numéro du bureau de vote **num_bureau**
- Le dictionnaire des candidats avec leur nombre de votes associé

Cette fonction enregistrera dans un fichier **{ville}_{num_bureau}.txt** les résultats du bureau de vote au format suivant :

Candidat 1 | 250

Candidat 2 | 3250

Candidat 3 | 890

Nul | 12

Blanc | 120

Si le fichier existe déjà, une exception sera levée pour indiquer cette anomalie et proposer une alternative.
Remarque : Pour mémoire, la fonction **os.listdir()** retourne la liste des fichiers dans le répertoire courant.

1.4. Analyser les résultats

On veut afficher nos résultats sous forme de pourcentages.

1.4.a) Calcul du nombre de votants

Écrire une fonction **nombre_votants()** qui prend en paramètre le dictionnaire des candidats et qui retourne un tuple comprenant le nombre de votants et le nombre de suffrages exprimés (nombre de votants - nuls - blancs).

1.4.b) Affichage des résultats

Écrire une fonction **afficher_resultats()** qui prend en paramètre le dictionnaire des votants. Cette fonction affichera pour chaque candidat son nom et le pourcentage de votes exprimés en sa faveur. Le pourcentage de votes en faveur d'un candidat se calcule sur le nombre de suffrages exprimés. Le pourcentage de votes nuls ou blancs se calcule sur le nombre total de votants.

Affichage attendu :

Total votants = 1000

Total exprimés = 90% (900)

Blancs = 7.5% (75)

Nuls = 2.5% (25)

Candidat 1 = 50% (450)

Candidat 2 = 30% (270)

Candidat 3 = 20% (180)

1.5 Programme principal

Écrire le programme principal qui effectue le dépouillement du bureau de vote n°19 de Compiègne pour le 1er tour des présidentielles, sauve les résultats dans un fichier et les affiche à l'écran.

Exercice 2 - Un gestionnaire de tâches (copie n°2)

2.1 Des tâches

Pour développer un outil de gestion de projets, on représente des tâches à exécuter dans le projet par des objets de la classe **Task**. Cette classe comporte un attribut de type **str** représentant l'identificateur d'une tâche et un attribut de type **int** représentant la durée en minutes pour exécuter la tâche. Le constructeur de cette classe a 2 paramètres qui permettent d'initialiser les attributs d'un objet après avoir vérifié la validité des arguments (types corrects attendus, durée positive). Si les arguments sont invalides, le constructeur déclenche une exception. La classe possède des accesseurs en lecture pour connaître les valeurs de ces attributs. Elle possède aussi un accesseur en écriture qui permet de modifier la durée de la tâche (si la donnée fournie est invalide, cette méthode déclenche une exception). Lorsqu'on exécute la commande **print(t)** où **t** est un objet **Task** d'identificateur **ID** et de durée **X**, on obtient l'affichage :

ID : duree=X

Écrire la classe **Task** ainsi que toutes ses méthodes afin d'obtenir le comportement expliqué ci-dessus. On fera attention à respecter le principe d'encapsulation.

Remarque : On remarquera qu'ici ce n'est pas la définition d'une méthode **print()** pour la classe **Task** qui est demandée (sinon, on exigerait un appel qui s'écrirait **t.print()**).

2.2 Des tâches prioritaires

Parmi les objets **Task**, certains ont aussi (en plus des attributs déjà mentionnés) un attribut de type **int**, prenant une valeur entre 0 et 100, et représentant une priorité (dans le gestionnaire de tâches, les tâches les plus prioritaires seront d'abord exécutées). Ces objets sont des instances de classe **PriorityTask**. Le constructeur de cette classe a 3 paramètres permettant d'initialiser ces 3 attributs. En plus des mêmes méthodes proposées par la classe **Task**, la classe **PriorityTask** possède aussi des accesseurs en lecture et écriture qui permettent de lire et modifier la priorité. La modification avec une donnée invalide provoque une exception. Lorsque l'on exécute la commande **print(t)** où **t** est un objet **PriorityTask** d'identificateur **ID**, de durée **X** et de priorité **Y**, on obtient l'affichage :

ID : duree=X, priorité=Y

Écrire la classe **PriorityTask** (qui hérite de **Task**) ainsi que toutes ses méthodes afin d'obtenir le comportement expliqué ci-dessus. On utilisera au mieux l'héritage.

2.3 Gestion des contraintes de précédences

Il peut y avoir des dépendances entre certaines tâches. En particulier, on peut avoir la contrainte qu'une tâche **A** (de type **Task** ou **PriorityTask**) doit être terminée avant de pouvoir commencer une autre tâche **B** (de type **Task** ou **PriorityTask**). On appelle cela une contrainte de précédence.

Un objet de la classe **Constraint** enregistre de telles contraintes dans un attribut de type **dict** dont les clés sont des valeurs de type **str** correspondant à des identificateurs de tâches, et dont les valeurs sont des listes contenant des identificateurs d'autres tâches (non nécessairement ordonnée). Par exemple, si le dictionnaire contient la valeur **["T4", "T2", "T5"]** associée à la clé **"T3"**, cela signifie que la tâche d'identificateur **"T3"** doit être terminée avant d'exécuter les tâches d'identificateur **"T4"**, **"T2"** et **"T5"**.

Le constructeur de cette classe ne possède aucun paramètre. Initialement, un objet **Constraint** ne contient aucune contrainte de précédence. La méthode **add_constraint(id1, id2)** permet d'ajouter une contrainte de précédence entre la tâche d'identificateur **id1** et la tâche d'identificateur **id2**. Cette méthode déclenche une exception si cette contrainte a déjà été ajoutée auparavant. La méthode **print_constraint("ID")** permet d'afficher **"ID ->"** suivi des tâches qui doivent être exécutées après la tâche d'identificateur **"ID"**. Cette méthode affiche **"ID -> _"** si la tâche d'identificateur **"ID"** ne précède aucune tâche.

Écrire la classe **Constraint** ainsi que toutes ses méthodes afin d'obtenir le comportement expliqué ci-dessus.

Utilisation de la classe **Constraint**

Écrire le code qui crée une instance de la classe **Constraint** et l'utilise de telle manière à provoquer l'affichage suivant :

T2 -> _

T3 -> T4 T2